ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 10

Хранение токенов на фронтенде

Рассмотрим способы хранения токенов на фронтенде, познакомимся с HTTP-заголовками (Headers) и файлами Cookies. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Способы хранения токенов

После того как сервер выдал пользователю JWT-токен, возникает вопрос, где его хранить на стороне клиента. От выбранного способа хранения напрямую зависит безопасность приложения. Наиболее распространёнными вариантами являются хранение токенов в **localStorage** или в **cookies**.

Хранение токенов в localStorage

`localStorage` — это механизм браузера для хранения данных в виде пар ключ–значение. Данные сохраняются между сессиями и доступны через JavaScript. Такой способ часто используется в одностраничных приложениях (SPA), где клиентская часть активно взаимодействует с API.

Сохранение токена может выглядеть следующим образом:

```
localStorage.setItem("accessToken", token);
```

Позже приложение может получить токен и использовать его при отправке запросов:

```
const token = localStorage.getItem("accessToken");

fetch("/api/profile", {
  headers: {
    Authorization: `Bearer ${token}`
```

```
  }
});
```

Основное преимущество `localStorage` заключается в простоте использования. JavaScript-код приложения может легко получить токен и добавить его в заголовок `Authorization`. Такой подход хорошо подходит для работы с REST API и позволяет гибко контролировать отправку токена.

Однако у этого метода есть серьёзный недостаток — уязвимость к **XSS-атакам (Cross-Site Scripting)**. Если злоумышленник сможет выполнить вредоносный JavaScript-код на странице, он сможет получить доступ к содержимому `localStorage` и украсть токен. После этого атакующий сможет отправлять запросы к API от имени пользователя.

Поэтому при использовании `localStorage` необходимо уделять особое внимание защите от XSS: корректной фильтрации данных, использованию безопасных библиотек и настройке политики безопасности контента (Content Security Policy).

Хранение токенов в cookies

Другим способом хранения токенов являются **cookies**. Cookies — это небольшие данные, которые браузер автоматически отправляет серверу вместе с каждым HTTP-запросом к соответствующему домену.

Сервер может установить cookie следующим образом:

```
res.cookie("accessToken", token, {
  httpOnly: true,
  secure: true,
  sameSite: "Strict"
});
```

После установки такой cookie браузер будет автоматически отправлять её при каждом запросе к серверу.

Важной особенностью cookies является возможность настройки параметров безопасности. Например, параметр `HttpOnly` делает cookie недоступной для JavaScript-кода. Это означает, что даже если на странице выполнится вредоносный скрипт, он не сможет получить токен. Параметр `Secure` заставляет браузер передавать cookie только по защищённому соединению HTTPS, что предотвращает перехват токена в сети. Параметр `SameSite` ограничивает отправку cookie в запросах, инициированных сторонними сайтами, что помогает защититься от атак типа CSRF.

Тем не менее использование cookies также имеет свои особенности. Поскольку браузер автоматически отправляет cookies с каждым запросом, сервер должен учитывать риск **CSRF-атак**. В таких случаях обычно применяются дополнительные механизмы защиты, например CSRF-токены.

Сравнение подходов

Хранение токенов в `localStorage` удобно и широко используется в клиентских приложениях, но требует хорошей защиты от XSS. Cookies позволяют лучше защитить токен от доступа через JavaScript благодаря параметру `HttpOnly`, однако при их использовании необходимо учитывать риск CSRF-атак.

В современных веб-приложениях часто применяется комбинированный подход. Короткоживущий **access-токен** используется для доступа к API, а более долгоживущий **refresh-токен** хранится в защищённой `HttpOnly` cookie. Когда срок действия access-токена истекает, клиент обращается к серверу с refresh-токеном и получает новую пару токенов.

Автоматизация работы с токенами

Для того, чтобы автоматизировать работу с токенами на фронтенде, удобно использовать встроенную в библиотеку `axios` утилиту: `interceptors`.

Interceptor (он же перехватчик) предназначен для предварительно обработки запросов и ответов от сервера.

Предположим у нас есть какой-то базовый HTTP-клиент:

```
import axios from "axios";

const apiClient = axios.create({
    baseURL: "https://curator.alekhin.moscow/api",
    headers: {
        "Content-Type": "application/json",
        "accept": "application/json",
    }
});
```

Ниже приведён пример автоматической подстановки access-токена в каждый запрос к серверу:

```
apiClient.interceptors.request.use(
    (config) => {
        let accessToken = localStorage.getItem('accessToken');
        if (accessToken) {
            config.headers.Authorization = `Bearer ${accessToken}`
        }
        return config;
    },
    (error) => Promise.reject(error)
);
```

Таким образом с помощью объекта `config` мы можем настраивать отправляемый запрос.

Это не единственный плюс перехватчиков. Также можно автоматически отправлять запросы на обновлении токена при истечении времени их действия:

```
apiClient.interceptors.response.use(
    (response) => response,
    async (error) => {
        let accessToken = localStorage.getItem('accessToken');
        let refreshToken = localStorage.getItem('refreshToken');
        let originalRequest = error.config;

        if (error.response.status === 401 && !originalRequest._retry) {
            originalRequest._retry = true;
            if (!accessToken || !refreshToken) {
                localStorage.removeItem('accessToken');
                localStorage.removeItem('refreshToken');
                return Promise.reject(error);
            }
            try {
                let response = await api.refresh(refreshToken);
                let isRefreshExpired = response.data.refresh_expired;
                if (isRefreshExpired) {
                    localStorage.removeItem('accessToken');
                    localStorage.removeItem('refreshToken');
                    return Promise.reject(error);
                }
                let newAccessToken = response.data.accessToken;
                let newRefreshToken = response.data.refreshToken;
                originalRequest.headers.Authorization = `Bearer
${newAccessToken}`;
                localStorage.setItem('accessToken', newAccessToken);
                localStorage.setItem('refreshToken', newRefreshToken);
                return apiClient(originalRequest);
            }
            catch (refreshError) {
                localStorage.removeItem('accessToken');
                localStorage.removeItem('refreshToken');
                return Promise.reject(refreshError);
            }
        }
        return Promise.reject(error)
    }
);
```

Логика следующая:

1. Если сервер ответил ошибкой с кодом 401, проверяем наличие токенов в локальном хранилище
2. Если одного из токенов нет в принципе - позволяем сработать ошибке (что должно привести к вводу учётных данных пользователем заново)
3. Если токены всё же есть, пробуем обновить access-токен
4. Если обновить получилось, повторно отправляем запрос с новым токеном
5. Если при обновлении произошла ошибка, позволяем ей сработать (что также должно привести к вводу учётных данных пользователем)

Практическое задание

Необходимо доработать задание из практического занятия №9: реализуйте фронтенд-часть приложения с использованием React.js или Vue.js (на выбор) и свяжите её с серверной частью.

Минимальный список маршрутов приложения:

Маршрут	Метод	Описание
/api/auth/register	POST	Регистрация (создание) пользователя
/api/auth/login	POST	Вход в систему
/api/auth/refresh	POST	Обновление пары токенов
/api/auth/me	GET	Получение информации о текущем пользователе
/api/products	POST	Создать товар
/api/products	GET	Получить список товаров
/api/products/:id	GET	Получить товар по id
/api/products/:id	PUT	Обновить параметры товара
/api/products/:id	DELETE	Удалить товар

Клиентская часть (фронтенд) должна позволять управлять товарами (просматривать список, создавать новый товар, просматривать детальную информацию о товаре по id, обновлять товар по id, удалять товар по id). Для входа в систему пользователь обязательно должен быть аутентифицирован (реализовать страницы входа и регистрации).

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой. Ссылка подгружается в соответствующий раздел СДО: Задания для самостоятельной работы -> Практические занятия 9-10.